Ajman University

College of Engineering and Information Technology

Department of Information Technology

Uniclass Scheduler

Automated University Course Scheduling System

By:

Shahem Fadi — 202210772 — Department of Information Technology

Omar Ali — 202211652 — Department of Information Technology

Othman Alaliy — 202211761 — Department of Information Technology

Mohamad Ahmad — 202210251 — Department of Information Technology

Supervised by:

Dr. Mahmoud Hammad

Project Submitted in partial fulfillment for the degree of Bachelor of Information Systems

Spring 2025–2026

Acknowledgment

We would like to express our sincere gratitude and appreciation to our supervisor, Dr. Mahmoud Hammad, for his invaluable guidance, continuous support, and constructive feedback throughout the development of this project. His expertise and dedication have been instrumental in shaping the direction and quality of our work.

We also extend our appreciation to the Department of Information Technology at Ajman University for providing the resources and academic environment necessary to carry out this project successfully.

Special thanks go to our families and friends for their unwavering moral support during the course of this project. Their patience and encouragement have been a source of motivation for all team members.

Finally, we would like to acknowledge the open-source communities behind the tools and technologies we used — FastAPI, React, OR-Tools, PostgreSQL, and others — whose contributions made our system possible.

Abstract

Uniclass Scheduler is a full-stack university management system designed to automate the process of generating academic course schedules. Manual timetable creation in universities is a time-consuming and error-prone task that often leads to conflicts in room assignments, instructor availability, and student enrollment.

This project addresses these challenges by developing an intelligent scheduling engine using Google OR-Tools, a constraint-satisfaction and optimization library. The system provides a role-based web interface built with React 19 and TypeScript, backed by a FastAPI (Python) REST API and a PostgreSQL relational database.

Administrators can input course sections, instructor assignments, and room capacities, then trigger an automated schedule generation process. The system produces conflict-free timetables and supports exporting results in PDF, DOCX, and Excel formats. Testing results confirm that the system can generate optimal schedules for realistic university datasets within acceptable time limits.

List of Abbreviations

API: Application Programming Interface

CSP: Constraint Satisfaction Problem

CRUD: Create, Read, Update, Delete

DB: Database

DOCX: Microsoft Word Open XML Document

ER: Entity Relationship

ERD: Entity Relationship Diagram

FastAPI: Fast Application Programming Interface (Python Web Framework)

HTTP: Hypertext Transfer Protocol

IS: Information Systems

IT: Information Technology

JWT: JSON Web Token

ORM: Object Relational Mapper

OR-Tools: Operations Research Tools (Google)

PDF: Portable Document Format

REST: Representational State Transfer

SDLC: Software Development Life Cycle

SQL: Structured Query Language

SWOT: Strengths, Weaknesses, Opportunities, Threats

UI: User Interface

UX: User Experience

Table of Contents

Table of Figures

Table of Tables

Chapter One: Introduction

This chapter introduces the Uniclass Scheduler project, explaining its name, background, the problem it addresses, and the objectives it seeks to achieve. It also summarises the project's main contributions and provides an overview of the documentation structure.

1.1 Background

Academic scheduling is one of the most complex administrative tasks in higher education institutions. A university must assign hundreds of course sections to classrooms, time slots, and instructors — all while satisfying a large number of hard and soft constraints. Hard constraints include avoiding room double-booking, preventing instructor schedule conflicts, and matching room capacities to enrolment figures. Soft constraints include instructor preferences, student convenience, and balanced workload distribution.

Traditionally, this process is carried out manually by academic coordinators and often requires several weeks of effort. The resulting schedules are frequently suboptimal and prone to human error. Uniclass Scheduler was conceived to replace this manual process with an automated, constraint-based intelligent system that produces valid, optimised timetables in a matter of seconds.

The system is named Uniclass Scheduler — combining 'University' and 'Class' — to reflect its targeted domain and purpose. It is built as a full-stack web application accessible to administrative staff through any modern browser.

1.2 Motivation and Problem Statement

The problem of automated timetable generation has been classified as an NP-hard combinatorial optimisation problem. Universities with large numbers of courses, instructors, and rooms face an exponential search space when attempting to build conflict-free schedules. Without computational support, administrators resort to error-prone manual approaches that are neither scalable nor reproducible.

The motivation for this project arises from real challenges observed in university administration:

- Scheduling conflicts lead to double-booked rooms and instructors.
- Manual scheduling takes weeks and still produces suboptimal results.
- Any change in room availability or instructor assignment requires re-doing large portions of the schedule.
- There is no centralised, digital, and exportable record of the schedule.

1.3 Objectives

The specific objectives of this project are:

1. Design and implement a constraint-based automated scheduling engine using Google OR-Tools.

2. Develop a secure, role-based web application with an administrator portal.
3. Enable administrators to input course sections, rooms, and instructor assignments.
4. Generate conflict-free timetables automatically upon request.
5. Display generated schedules in an interactive weekly timetable grid.
6. Allow export of schedules to PDF, DOCX, and Excel formats.
7. Provide detailed suggestions for handling unscheduled sections.

1.4 Project Contributions

The main contributions of this project include:

- Constraint-Based Solver Integration: Integration of Google OR-Tools into a web-based university management system, demonstrating practical application of operations research in higher education.
- Full-Stack Architecture: A clean separation between a Python FastAPI backend and a React TypeScript frontend, following modern software-engineering best practices.
- Multi-Format Export: The ability to export schedules to PDF, DOCX, and Excel, supporting the workflow of academic administrators.
- Unscheduled Section Reporting: The system identifies sections that could not be scheduled and presents actionable suggestions for resolution.
- Reusable Platform: The system is designed to be extensible and adaptable to different universities with varying scheduling rules.

1.5 Documentation Overview

This documentation is organised as follows:

- Chapter One (Introduction): Project background, motivation, objectives, and contributions.
- Chapter Two (Literature Review): Related work, key concepts, SWOT analysis, and market comparison.
- Chapter Three (Planning and Requirements): Scope, roles, risks, methodology, and functional/non-functional requirements.
- Chapter Four (Analysis and Design): System architecture, use cases, diagrams, ER model, and UI design.
- Chapter Five (Implementation): Technology stack, testing strategy, and deployment plan.
- Chapter Six (Conclusion): Strengths, weaknesses, future improvements, and closing remarks.
- References: Cited works and resources.
- Appendix: Supporting materials and additional figures.

Chapter Two: Literature Review

This chapter reviews the existing body of knowledge related to automated scheduling systems, constraint-satisfaction techniques, and comparable tools in the academic scheduling domain.

2.1 Automated Scheduling Systems

Academic timetabling has been an active area of research since the 1960s. Early approaches relied on graph-colouring algorithms to model scheduling as a vertex-colouring problem. Carter & Laporte (1996) provided a comprehensive survey classifying timetabling problems into examination timetabling and course timetabling, highlighting the NP-hard nature of both categories.

More recent research has focused on metaheuristic approaches such as genetic algorithms, simulated annealing, and tabu search. However, these offer approximations and may not guarantee constraint satisfaction. Constraint Programming (CP) and Integer Linear Programming (ILP) provide provably optimal solutions for smaller problem instances and have become practical as computing power has increased.

2.2 Constraint Satisfaction Problems (CSP)

A Constraint Satisfaction Problem (CSP) is defined by a set of variables, a domain for each variable, and a set of constraints restricting valid combinations of assignments. Academic scheduling maps naturally to this formulation: variables are course-section assignments, domains are time slots and rooms, and constraints encode rules such as 'no two courses can share the same room at the same time'.

Google OR-Tools' CP-SAT solver uses a combination of SAT-based techniques and constraint propagation to explore feasible assignments efficiently. It supports both hard constraints (which must be satisfied) and objective functions (which the solver minimises), enabling the optimisation of soft preferences such as instructor workload balance. This project applies OR-Tools by encoding room-capacity constraints, instructor availability, and time-slot uniqueness as hard constraints.

2.3 SWOT Analysis

Strengths • Fully automated conflict-free scheduling • Uses proven OR-Tools CP-SAT solver • Multi-format export (PDF, DOCX, Excel) • Modern React-based interface • Open-source stack — no licensing cost	**Weaknesses** • Single admin role; no student/faculty portal • No real-time notification system • Scalability with 1000+ sections untested • Manual data entry for initial setup
Opportunities • Expandable to multi-campus environments • Integration with student registration systems • Growing demand for digital academic tools • Mobile application extension	**Threats** • Competition from commercial ERP systems • Changing university policies may require reconfiguration • Security risks if not maintained

Table 3: SWOT Analysis

2.4 Market Analysis

The following table compares Uniclass Scheduler with notable alternatives:

System	Type	Scheduling Engine	Export	Cost
Uniclass Scheduler	Open-source	Google OR-Tools (CP-SAT)	PDF, DOCX, Excel	Free
FET	Open-source	Genetic Algorithm	HTML, PDF	Free
UniTime	Open-source	ILP / Sim. Annealing	PDF, iCal	Free (complex)
Scientia Syllabus+	Commercial	Proprietary	PDF, Excel	Licensed
EMS Campus	Commercial	Proprietary	Multiple	Licensed

Table 4: Market Analysis Comparison

Chapter Three: Planning and Requirements

3.1 Project Scope

Uniclass Scheduler targets university administrative staff responsible for academic scheduling. The system allows administrators to manage courses, sections, rooms, and instructors, then automatically generate timetables. The current scope covers the admin portal, automated schedule generation, timetable visualisation, and multi-format export.

3.2 Project Scenarios and Plan

3.2.1 Roles and Responsibilities

Member	Student ID	Role
Shahem Fadi	202210772	Team Lead, Backend Developer (FastAPI, OR-Tools)
Omar Ali	202211652	Frontend Developer (React, TypeScript)
Othman Alaliy	202211761	Database Designer (PostgreSQL, SQLAlchemy)
Mohamad Ahmad	202210251	Testing, Documentation, Export Module

Table 2: Roles and Responsibilities

3.2.2 Constraints

- Time Constraint: The project must be completed within one academic semester.
- Technology Constraint: Must use open-source technologies compatible with the Replit deployment environment.
- User Constraint: The current version targets admin users only; student-facing features are out of scope.
- Data Constraint: Initial data entry is performed manually; integration with existing student information systems is not included.

3.2.3 Risk Assessment

Risk	Likelihood	Impact	Mitigation
OR-Tools fails to find feasible solution	Medium	High	Design fallback reporting for unsatisfiable constraints
Database performance under load	Low	Medium	Indexing and query optimisation
JWT token security vulnerabilities	Low	High	Short token expiry and HTTPS enforcement
Team member unavailability	Medium	Medium	Cross-training and shared code documentation

Browser compatibility issues	Low	Low	Standard React/Vite build tools

Table 1: Risk Analysis

3.2.4 Project Management Approach / Methodology

The project follows an Agile/Scrum methodology, organised into two-week sprints. Each sprint targets specific deliverables: backend API endpoints, frontend UI components, or solver integration. A shared GitHub repository is used for version control and collaboration, with regular meetings to review progress and adjust plans.

The SDLC phases followed are: Requirements Analysis → System Design → Implementation → Testing → Deployment → Documentation. This iterative process ensures that feedback is incorporated continuously and the final product meets the specified requirements.

3.3 Functional Requirements

- The system shall allow administrators to log in using a username and password.
- The system shall allow administrators to manage course sections (add, edit, delete).
- The system shall allow administrators to manage rooms, including capacity and availability.
- The system shall allow administrators to assign instructors to course sections.
- The system shall generate a conflict-free weekly schedule automatically using OR-Tools.
- The system shall display the generated schedule in an interactive weekly timetable grid.
- The system shall identify and display unscheduled sections with actionable suggestions.
- The system shall export the schedule in PDF, DOCX, and Excel formats.
- The system shall support JWT-based authentication with role-based access control.

3.4 Non-Functional Requirements

- **Usability:** The interface must be intuitive for non-technical admin users with minimal training.
- **Performance:** Schedule generation must complete within 30 seconds for up to 200 course sections.
- **Reliability:** The system must maintain data integrity and recover gracefully from errors.
- **Security:** All API endpoints must be protected by JWT authentication; passwords must be hashed.
- **Portability:** The application must run on any modern browser without installation.

- **Scalability:** The architecture must support increasing numbers of courses and users without redesign.

Chapter Four: Project Analysis and Design

4.1 Context Diagram

The Uniclass Scheduler system interacts with two external entities: the Administrator (who inputs data and triggers schedule generation) and the Database (which stores all persistent entities). The system receives scheduling requests, processes them through the OR-Tools solver, and returns generated timetables and export files.

4.2 Use Case Diagram

The primary use cases for the system are:

- **UC-01 — Login:** Admin authenticates using credentials; system returns JWT token.
- **UC-02 — Manage Course Sections:** Admin adds, edits, or removes course sections.
- **UC-03 — Manage Rooms:** Admin defines room names, capacities, and availability.
- **UC-04 — Assign Instructors:** Admin assigns instructors to sections.
- **UC-05 — Generate Schedule:** Admin triggers automated schedule generation.
- **UC-06 — View Timetable Grid:** Admin views the generated schedule in a weekly grid.
- **UC-07 — View Unscheduled Sections:** Admin reviews sections that could not be scheduled.
- **UC-08 — Export Schedule:** Admin exports the schedule to PDF, DOCX, or Excel.

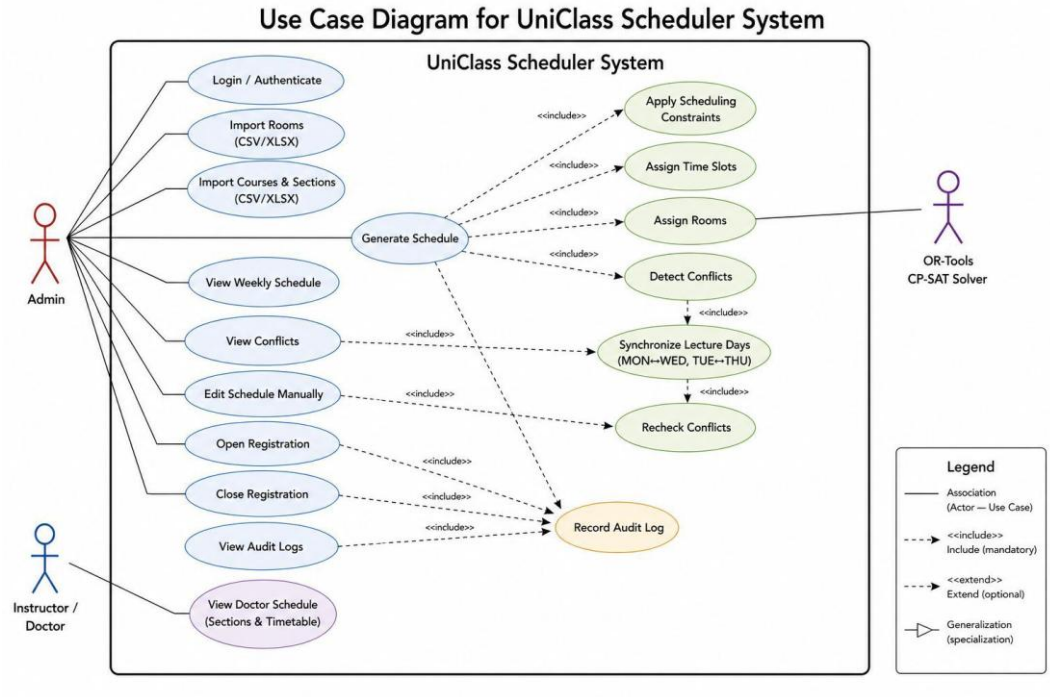


Figure 18: Use Case Diagram

4.3 Activity Diagram

The scheduling activity follows these steps: (1) Admin logs in, (2) Admin enters course, room, and instructor data, (3) Admin triggers schedule generation, (4) System encodes constraints into OR-Tools model, (5) Solver runs and returns assignments, (6) System stores results and displays timetable grid, (7) Admin reviews and exports.

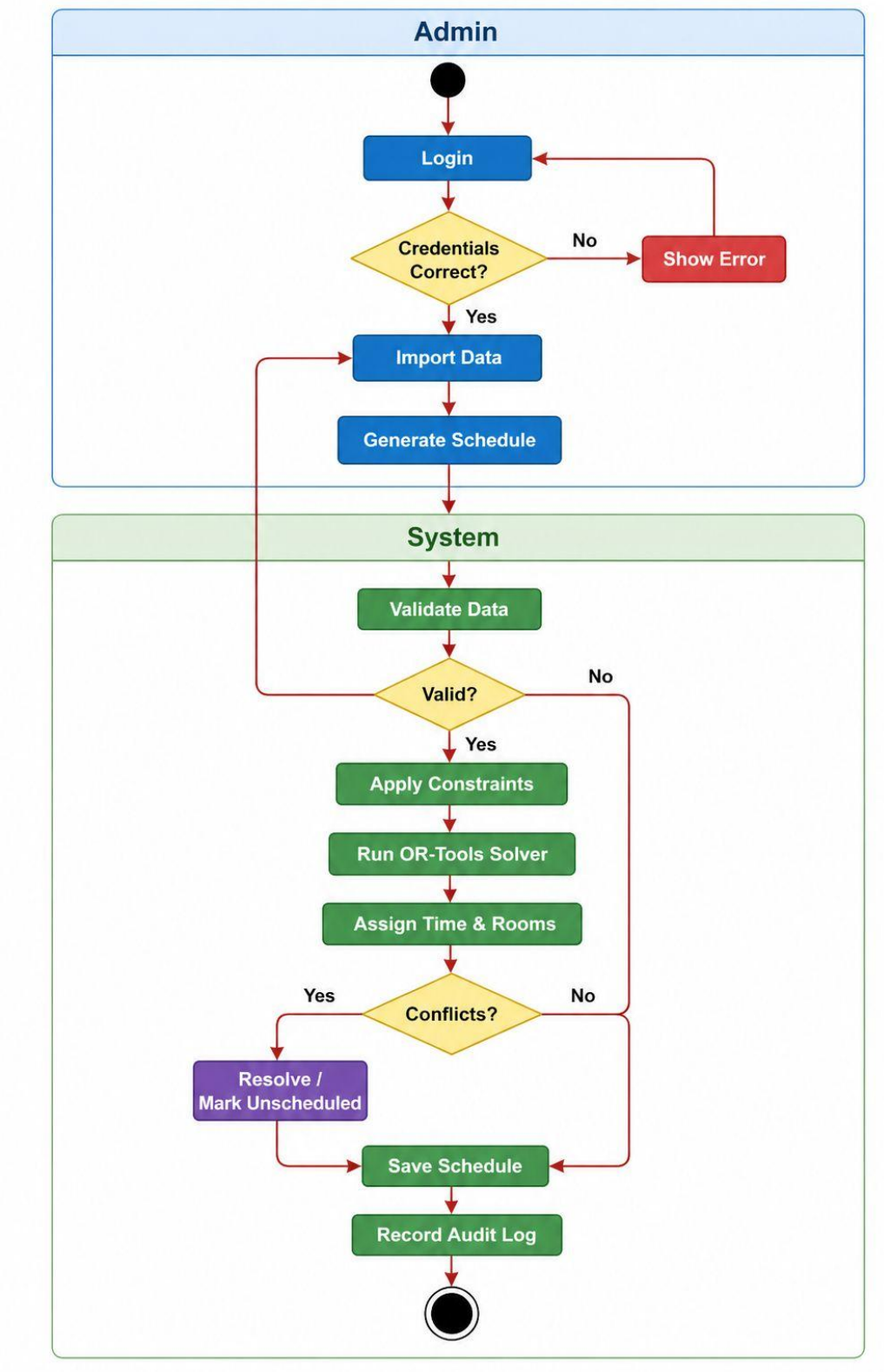


Figure 19: Activity Diagram

4.4 Sequence Diagram

The schedule-generation sequence involves the following interactions: the Admin sends a POST request to /api/v1/schedules/generate via the React frontend. The FastAPI server authenticates the request using the JWT token, loads course sections, rooms, and instructor assignments from PostgreSQL, passes them to the OR-Tools solver, and stores the resulting assignments. The server returns a success response, which the frontend uses to fetch and render the timetable grid.

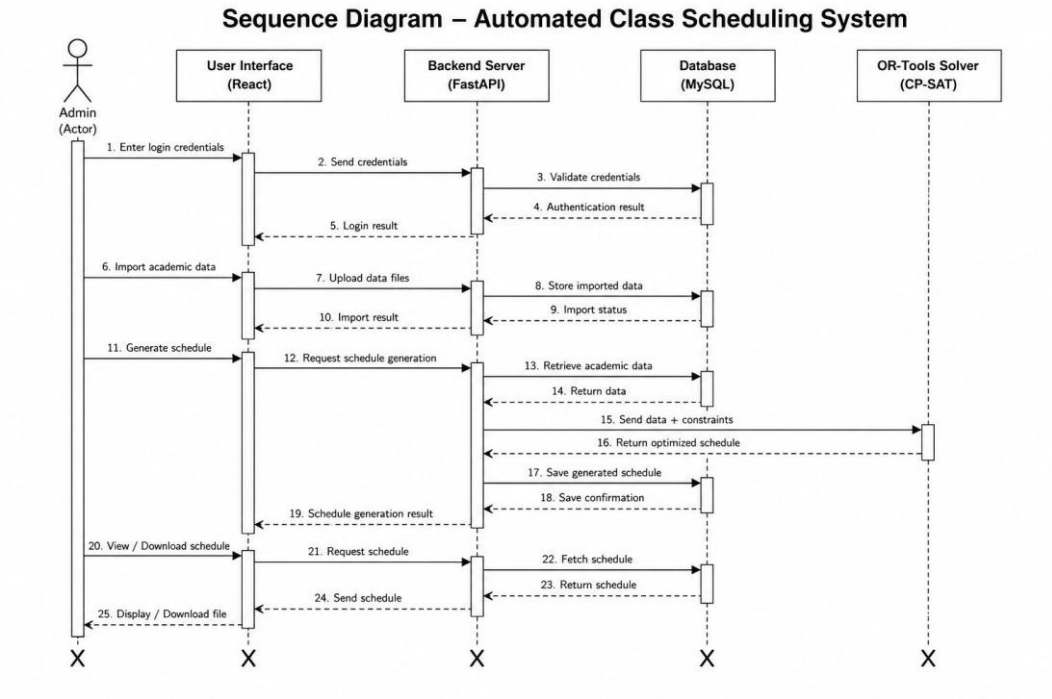


Figure 20: Sequence Diagram

4.5 ER Diagram

The database schema consists of the following entities and relationships:

- User (id, username, hashed_password, role)
- Course (id, code, name, credits)
- Section (id, course_id, instructor_id, enrollment_count)
- Room (id, name, capacity, building)
- TimeSlot (id, day_of_week, start_time, end_time)
- ScheduleEntry (id, section_id, room_id, time_slot_id, generated_at)

Relationships: A Course has many Sections. Each Section is assigned to one Instructor (User). A ScheduleEntry links a Section, a Room, and a TimeSlot, representing a final assignment in the generated timetable.

4.6 User Interface Design

The frontend is built with React 19 and TypeScript. The main screens include:

- **Login Page (/login):** Username/password form with JWT-based authentication and role-based redirect.
- **Admin Dashboard (/admin):** Overview panel showing scheduling statistics and quick actions.
- **Schedule Generator:** Button to trigger OR-Tools schedule generation with real-time status feedback.
- **Timetable Grid (/admin/schedule):** Interactive weekly grid view displaying all scheduled sections.
- **Unscheduled Sections Panel:** List of sections that could not be scheduled, with expandable suggestion details.
- **Export Controls:** Buttons to download the schedule as PDF, DOCX, or Excel.

4.7 Architectural Design

Uniclass Scheduler follows a three-tier architecture:

- **Presentation Tier:** React 19 + TypeScript + Vite frontend served on port 5000. Communicates with the backend via REST API calls.
- **Application Tier:** Python FastAPI server running on port 8000 via Uvicorn. Handles authentication (JWT), business logic, and OR-Tools solver invocation.
- **Data Tier:** PostgreSQL database accessed through SQLAlchemy ORM with Alembic for migrations.

4.8 Network Design

In the development environment, the React dev server proxies all requests to /api to the FastAPI server at http://127.0.0.1:8000. In production, both services are deployed on Replit, with the frontend served as a static build and the backend running as a persistent web service. HTTPS is enforced at the platform level, ensuring secure communication.

Chapter Five: Implementation

5.1 Environment — Technologies

The following technologies were used to implement Uniclass Scheduler:

Category	Technology	Notes
Backend Framework	FastAPI (Python)	Async REST API framework
Scheduling Solver	Google OR-Tools	CP-SAT constraint solver
Database	PostgreSQL	Relational database via Replit
ORM	SQLAlchemy	Python ORM with Alembic migrations
Frontend Framework	React 19 + TypeScript	Component-based UI
Build Tool	Vite	Fast frontend dev server and bundler
Authentication	JWT (python-jose)	Stateless token-based auth
Export	reportlab, python-docx, openpyxl	PDF, DOCX, Excel generation
Deployment	Replit	Cloud-hosted, PostgreSQL-integrated
Version Control	Git / GitHub	Collaborative development

Table 5: Technology Stack

5.2 Evaluation / Testing Measures

5.2.1 Testing Goals

The testing goal is to verify that the system correctly generates conflict-free schedules, handles edge cases (e.g., no feasible solution, instructor not available), and delivers a reliable user experience across all major admin workflows.

5.2.2 Testing Plans

- Define test cases for each use case (UC-01 through UC-08).
- Unit test each FastAPI route using pytest with a dedicated test database.
- Integration test the OR-Tools solver with known-solvable and known-unsolvable inputs.
- End-to-end test the full flow from login to schedule export.

5.2.3 Testing (Unit, Integration, System)

- **Unit Testing:** Individual FastAPI route handlers and Pydantic models are tested using pytest. Ruff is used for linting and code formatting to maintain code quality.

- **Integration Testing:** The OR-Tools solver is tested with datasets of 10, 50, and 100 course sections to verify correctness and performance benchmarks.
- **System Testing:** The complete application is tested end-to-end on the Replit deployment environment, verifying authentication, schedule generation, timetable display, and export functionality.

5.3 Implementation Screenshots

The following screenshots illustrate the key screens and features of the Uniclass Scheduler application.

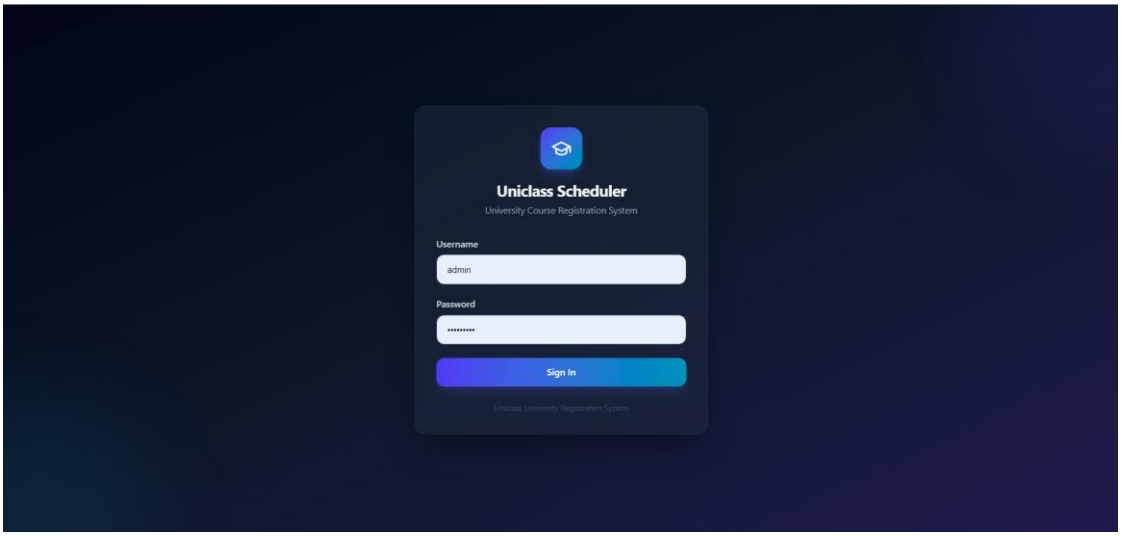


Figure 1: Login Page

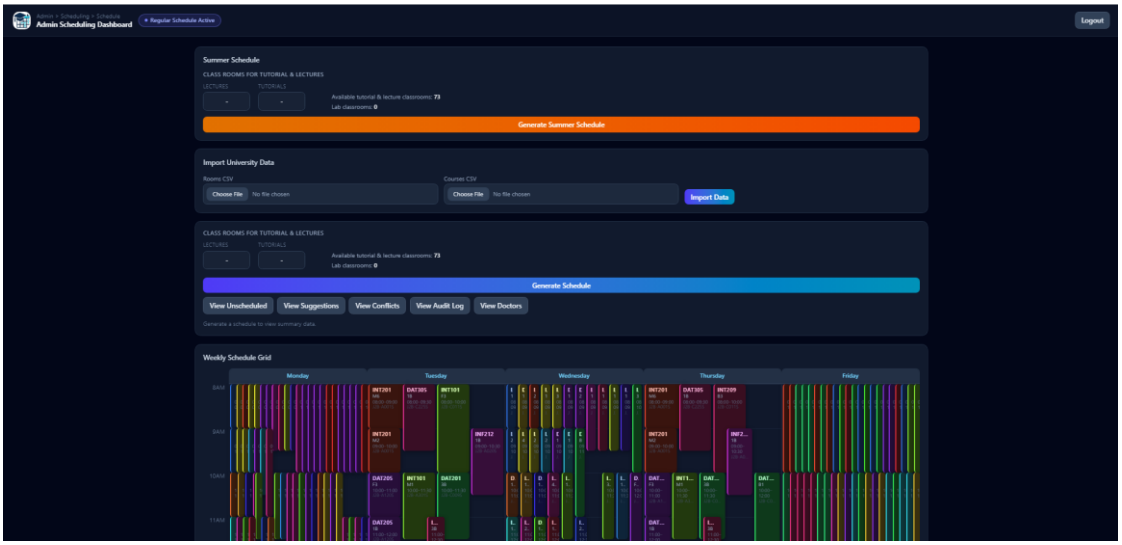


Figure 2: Admin Dashboard

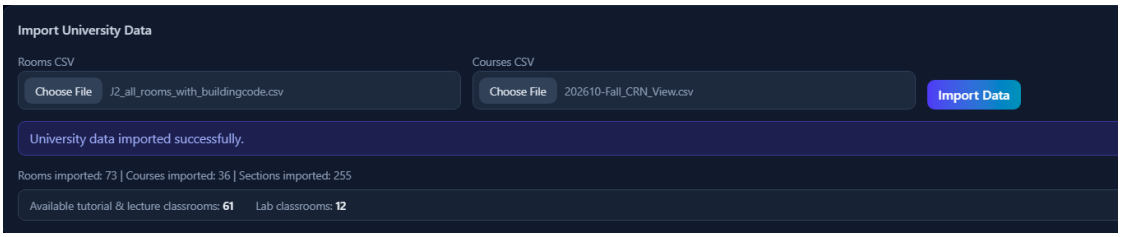


Figure 3: Importing Rooms and Courses

	A	B	C	D	E	F	G	H
1	Course Co	Course Na	Section Ty	Section	Doctor Nai	Doctor ID	Credits	Prerequisite
2	INT101	Calculus fc	Lecture	2B	Dr. Abdeaz	4898	3	none
3	INT101	Calculus fc	Lecture	3B	Dr. Abdeaz	4898	3	none
4	DAT100	Introductic	Lecture	1B	Dr. Abdela	8000	3	none
5	DAT100	Introductic	Lecture	2B	Dr. Abdela	8000	3	none
6	DAT304	Data Analy	Lecture	1B	Dr. Abdess	8001	3	DAT206
7	INT302	Database I	Lecture	2B	Dr. Abdess	8001	3	INT201
8	DAT205	Programm	Lecture	2M	Dr. Al Beta	5131	3	INT100
9	DAT401	Data Minin	Lecture	1B	Dr. Al Beta	5131	3	DAT302
10	INS415	Biomedica	Lecture	1B	Dr. Al Naw	6398	3	INT302,INT206
11	INT100	Introducto	Lecture	3B	Dr. Al Naw	6398	3	none
12	INT100	Introducto	Lecture	4B	Dr. Al Naw	6398	3	none
13	INT201	Object-Ori	Lecture	F3	Dr. Al Naw	6398	3	INT100
14	INT201	Object-Ori	Lecture	M2	Dr. Al Naw	6398	3	INT100
15	INT201	Object-Ori	Lecture	M6	Dr. Al Naw	6398	3	INT100
16	DAT305	Statistical	Lecture	1B	Dr. AlShbo	5522	3	DAT203
17	DAT405	Machine Le	Lecture	1B	Dr. AlShbo	5522	3	DAT401
18	DAT201	Linear Alge	Lecture	1B	Dr. AlShbo	5522	3	INT101
19	DAT201	Linear Alge	Lecture	2B	Dr. AlShbo	5522	3	INT101
20	DAT201	Linear Alge	Lecture	3B	Dr. AlShbo	5522	3	INT101
21	INS308	Enterprise	Lecture	1B	Dr. Ammar	4896	3	INT302
22	INS309	Knowledge	Lecture	1B	Dr. Ammar	4896	3	INT302
23	INS404	IS Strategy	Lecture	1B	Dr. Ammar	4896	3	INT307
24	INS413	Project Qu	Lecture	1B	Dr. Ammar	4896	3	INT307

Figure 4: Input Courses Screen

	A	B	C	D	E	F
6	J2M	C112M	C112M	50	lecture,tutorial	
7	J2B	A001S	A001S	80	lecture,tutorial	
8	J2B	C113S	C113S	48	lecture,tutorial	
9	J2B	C030S	C030S	50	lecture,tutorial	
10	J2B	A101S	A101S	80	lecture,tutorial	
11	J2F	C130F	C130F	55	lecture,tutorial	
12	J2M	A003M	A003M	55	lecture,tutorial	
13	J2B	A301S	A301S	66	lecture,tutorial	
14	J2B	C009S	C009S	80	lecture,tutorial	
15	J2F	A122F	A122F	55	lecture,tutorial	
16	J2M	A105M	A105M	58	lecture,tutorial	
17	J2B	B208S	B208S	55	lecture,tutorial	
18	J2B	A020S	A020S	80	lecture,tutorial	
19	J2F	1724	Computer	25	lab	
20	J2B	C225S	C225S	75	lecture,tutorial	
21	J2F	1729	Computer	25	lab	
22	J2B	A220S	A220S	48	lecture,tutorial	
23	J2F	C027F	C027F	55	lecture,tutorial	
24	J2B	C031S	C031S	48	lecture,tutorial	
25	J2F	A125F	A125F	50	lecture,tutorial	
26	J2B	A120S	A120S	80	lecture,tutorial	
27	J2B	C132S	C132S	80	lecture,tutorial	
28	J2B	A206S	A206S	110	lecture,tutorial	
29	J2B	C111S	C111S	80	lecture,tutorial	
30	J2F	A023F	A023F	55	lecture,tutorial	
31	J2F	A024F	A024F	55	lecture,tutorial	
32	J2F	A124F	A124F	60	lecture,tutorial	
33	J2B	C209S	C209S	55	lecture,tutorial	
34	J2M	A201M	A201M	70	lecture,tutorial	
35	J2B	C114S	C114S	80	lecture,tutorial	
36	J2F	A224F	A224F	50	lecture,tutorial	
37	J2M	A207M	A207M	50	lecture,tutorial	
38	J2M	A205M	A205M	60	lecture,tutorial	
39	J2F	A221F	A221F	55	lecture,tutorial	
40	J2F	A223F	A223F	80	lecture,tutorial	
41	J2M	B204M	B204M	60	lecture,tutorial	

Figure 5: Input Rooms Screen

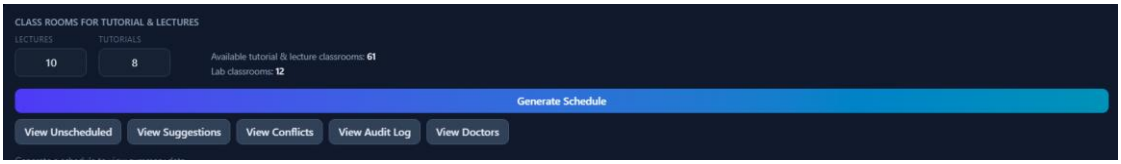


Figure 6: Room Configuration (Lecture / Tutorial / Lab)

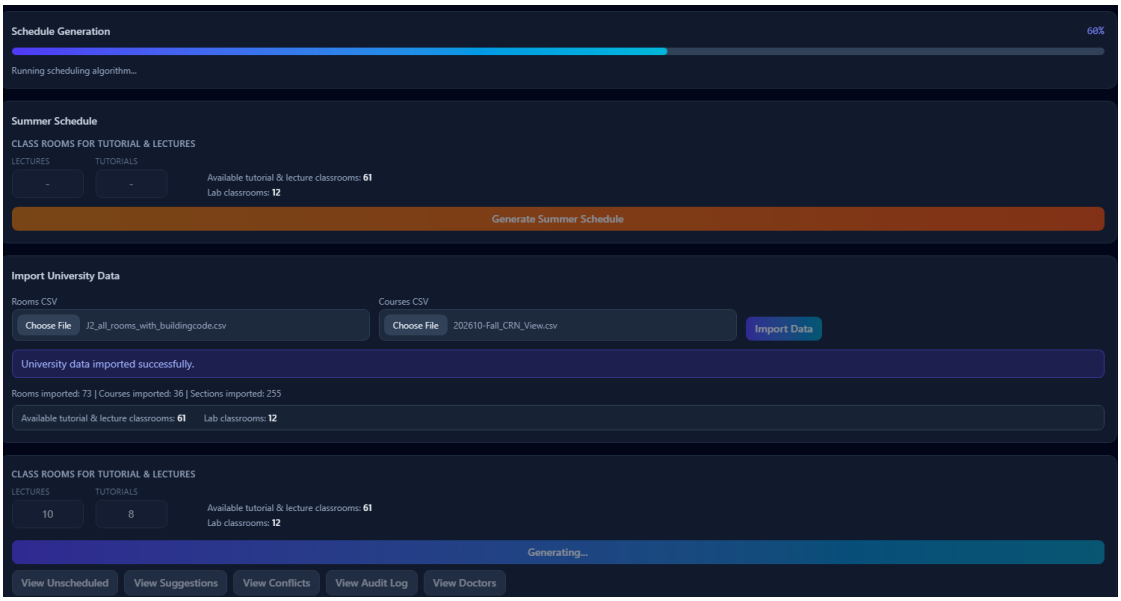


Figure 7: Press Generate and Wait

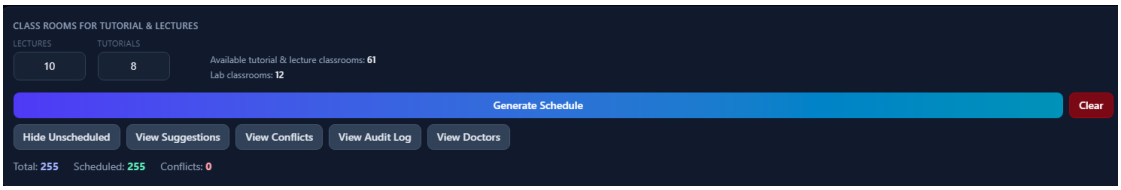


Figure 8: Schedule After Generation

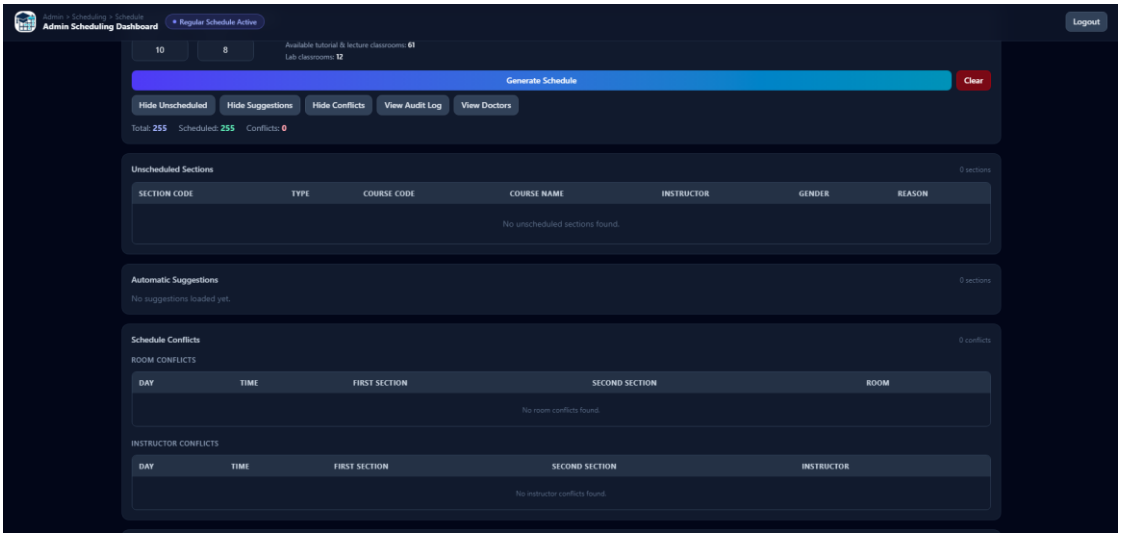


Figure 9: Unscheduled Sections and Conflict Suggestions

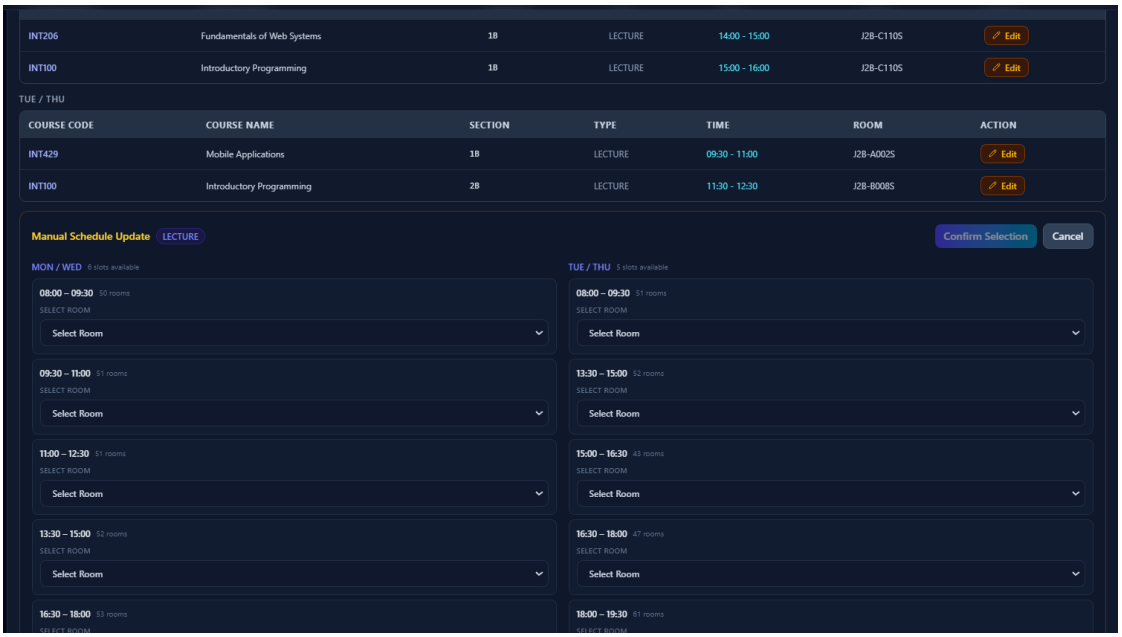


Figure 10: Manual Edit Feature

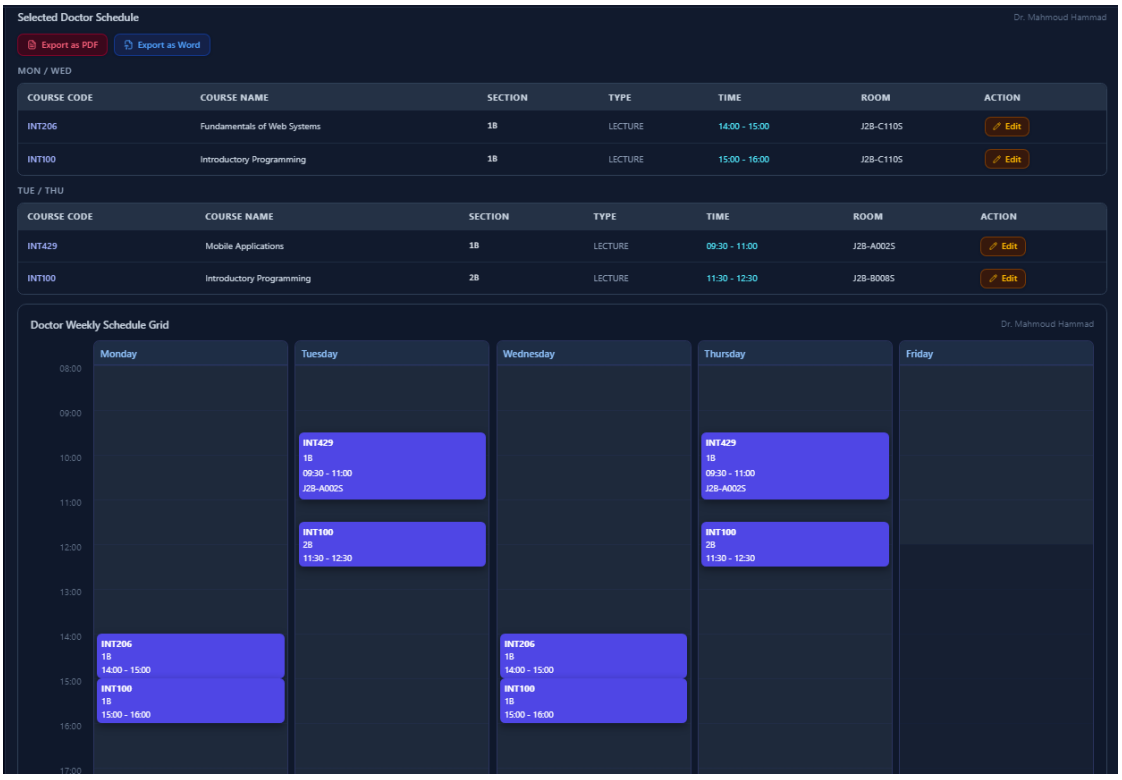


Figure 11: Instructor (Doctor) Schedule View

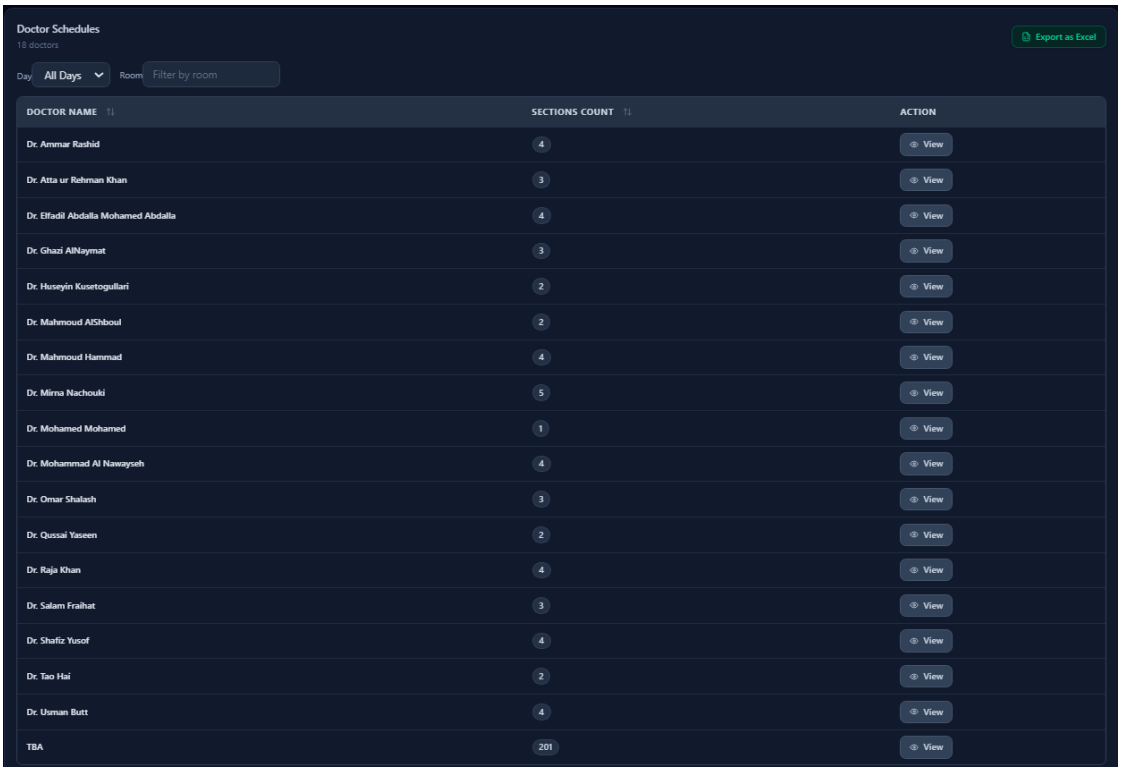


Figure 12: View Doctor Button

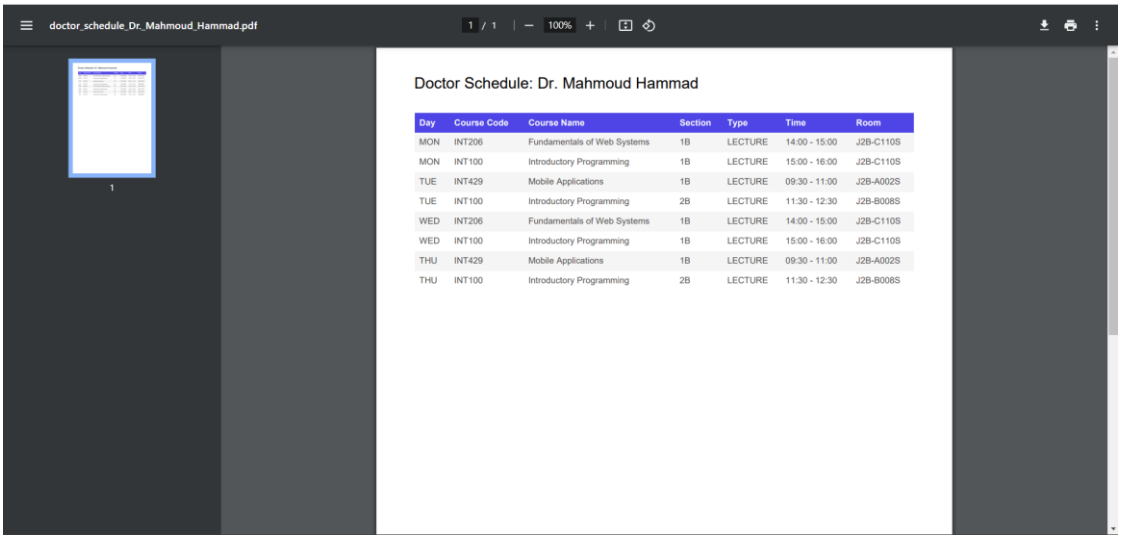
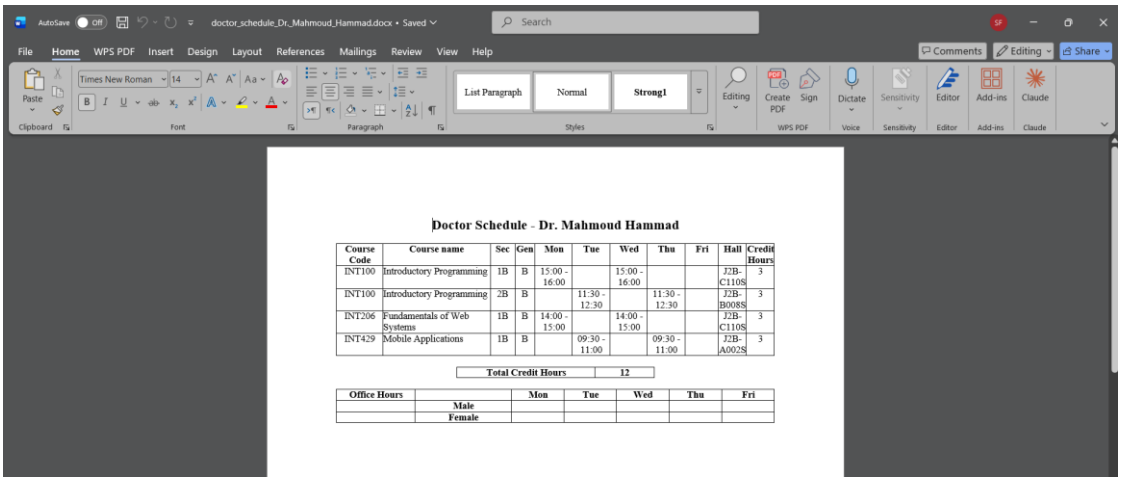


Figure 13: Export to PDF Feature



Doctor Schedule - Dr. Mahmoud Hammad

Course Code	Course name	Sec	Gen	Mon	Tue	Wed	Thu	Fri	Hall	Credit Hours
INT100	Introductory Programming	1B	B	15:00 - 16:00		15:00 - 16:00			J2B-C110S	3
INT100	Introductory Programming	2B	B		11:30 - 12:30		11:30 - 12:30		J2B-B008S	3
INT206	Fundamentals of Web Systems	1B	B	14:00 - 15:00		14:00 - 15:00			J2B-C110S	3
INT429	Mobile Applications	1B	B		09:30 - 11:00		09:30 - 11:00		J2B-A002S	3

Total Credit Hours	12

Office Hours		Mon	Tue	Wed	Thu	Fri
	Male					
	Female					

Figure 14: Export to Word Feature

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	College co	College de	Subject co	Course nu	Course tit	Credit hou	Schedule t	Crn	Section nu	Max capac	Day	Meeting ti	Room	Instructor	Merged
2	En	college of	INT	311	Advanced	3	lecture	10005	1B	50	TUE	14:00 - 15:	J2B-C030S	Dr. Atta ul	yes
3	En	college of	INT	311	Advanced	3	lecture	10006	2B	50	TUE	15:00 - 16:	J2B-C031S	Dr. Atta ul	yes
4	En	college of	INT	311	Advanced	3	lecture	10005	1B	50	THU	14:00 - 15:	J2B-C030S	Dr. Atta ul	yes
5	En	college of	INT	311	Advanced	3	lecture	10006	2B	50	THU	15:00 - 16:	J2B-C031S	Dr. Atta ul	yes
6	En	college of	INT	311	Advanced	0	lab	10212	2M	25	THU	16:00 - 18:	J2M-1625	TBA	no
7	En	college of	INT	311	Advanced	0	lab	10215	1F	25	THU	17:30 - 19:	J2F-1724	TBA	no
8	En	college of	INT	311	Advanced	0	lab	10216	1M	25	THU	17:30 - 19:	J2M-1658	TBA	no
9	En	college of	INT	311	Advanced	0	lab	10217	2F	25	THU	17:30 - 19:	J2F-1725	TBA	no
10	En	college of	INT	311	Advanced	0	lab	10218	3M	25	THU	17:30 - 19:	J2M-1627	TBA	no
11	En	college of	INT	430	Artificial I	3	lecture	10032	2B	50	MON	10:30 - 11:	J2B-B208S	Dr. Omar S	yes
12	En	college of	INT	430	Artificial I	3	lecture	10034	1B	50	TUE	15:00 - 16:	J2B-A220S	Dr. Omar S	yes
13	En	college of	INT	430	Artificial I	3	lecture	10032	2B	50	WED	10:30 - 11:	J2B-B208S	Dr. Omar S	yes
14	En	college of	INT	430	Artificial I	3	lecture	10034	1B	50	THU	15:00 - 16:	J2B-A220S	Dr. Omar S	yes
15	En	college of	INT	430	Artificial I	3	lecture	10092	3B	50	TUE	11:30 - 12:	J2B-C113S	TBA	yes
16	En	college of	INT	430	Artificial I	3	lecture	10092	3B	50	THU	11:30 - 12:	J2B-C113S	TBA	yes
17	En	college of	INT	430	Artificial I	0	lab	10248	1F	25	FRI	10:00 - 12:	J2F-1725	TBA	no
18	En	college of	INT	430	Artificial I	0	lab	10249	1M	25	FRI	10:00 - 12:	J2M-1624	TBA	no
19	En	college of	INT	430	Artificial I	0	lab	10250	2F	25	FRI	10:00 - 12:	J2F-1724	TBA	no
20	En	college of	INT	430	Artificial I	0	lab	10251	2M	25	FRI	10:00 - 12:	J2M-1658	TBA	no
21	En	college of	INT	430	Artificial I	0	lab	10252	3F	25	FRI	10:00 - 12:	J2F-1716	TBA	no
22	En	college of	INT	430	Artificial I	0	lab	10253	3M	25	FRI	10:00 - 12:	J2M-1625	TBA	no
23	En	college of	INT	430	Artificial I	0	lab	10254	4M	25	FRI	10:00 - 12:	J2M-1628	TBA	no

Figure 15: Export to Excel Feature

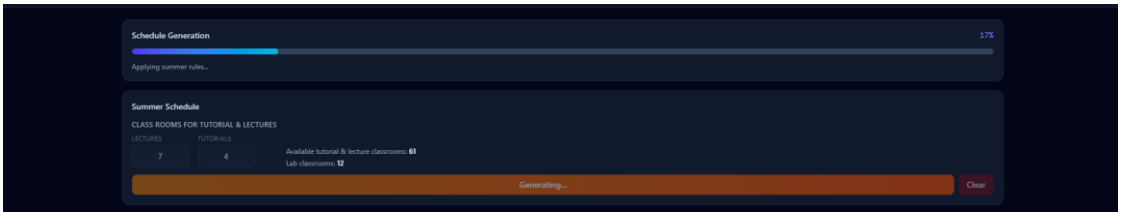


Figure 16: Summer Schedule Generation

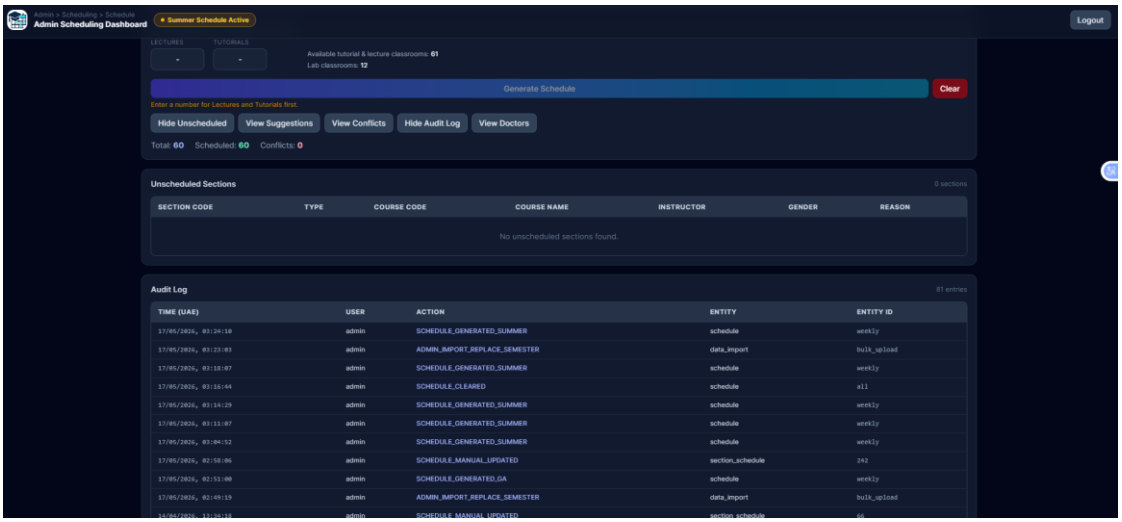


Figure 17: Schedule After Summer Generation

5.4 Implementation Schedule

Phase	Tasks	Duration
Phase 1 — Requirements	Gather requirements, define use cases, design ER diagram	Week 1–2
Phase 2 — Database & Backend	Design DB schema, implement FastAPI routes, set up authentication	Week 3–5
Phase 3 — Solver Integration	Integrate OR-Tools CP-SAT, define constraints, test solver	Week 6–7
Phase 4 — Frontend	Build React UI, timetable grid, admin dashboard	Week 8–10
Phase 5 — Export Module	Implement PDF, DOCX, Excel export using Python libraries	Week 11
Phase 6 — Testing	Unit, integration, and system testing; bug fixing	Week 12–13
Phase 7 — Deployment & Docs	Deploy on Replit, write documentation, prepare final report	Week 14–15

Table 6: Implementation Schedule

Chapter Six: Conclusion

6.1 Strengths and Weaknesses

Strengths:

- Fully automated conflict-free schedule generation using a proven constraint-satisfaction solver.
- Clean, modern React-based interface accessible to non-technical administrative users.
- Role-based access control with JWT authentication ensures data security.
- Multi-format export (PDF, DOCX, Excel) supports diverse administrative workflows.
- Transparent reporting of unscheduled sections with actionable suggestions.
- Open-source technology stack with no licensing costs.

Weaknesses:

- The current version supports a single administrative role; student and faculty self-service portals are not included.
- Initial data population is manual, requiring an import mechanism for large datasets.
- Solver performance for very large universities (1000+ sections) has not been benchmarked.
- No real-time collaboration or notification features for multi-user scenarios.

6.2 Improvements for the Future

- Student Portal: Allow students to view their personal timetables and check room availability.
- Instructor Portal: Enable instructors to submit availability preferences that feed into the solver.
- Data Import: Add CSV/Excel import functionality for bulk data entry of courses, rooms, and sections.
- Real-Time Notifications: Notify relevant parties when a schedule is generated or updated.
- Multi-Campus Support: Extend the room and scheduling model to support multiple physical campuses.
- Mobile Application: Build a companion mobile app for on-the-go schedule access.
- AI-Assisted Suggestions: Use machine learning to recommend instructor assignments and room allocations based on historical data.

6.3 Conclusion

The Uniclass Scheduler project successfully demonstrates the application of constraint-based optimisation to a real-world academic scheduling problem. By integrating Google OR-Tools with a modern full-stack web architecture, the system provides university administrators with a powerful, user-friendly tool for automating the time-consuming and error-prone task of timetable creation.

The project meets all stated objectives: it generates conflict-free schedules, provides a clean dashboard for administration, displays results in an interactive timetable grid, and supports multi-format export. The role-based authentication system ensures that administrative data is protected.

From a technical standpoint, the project demonstrates proficiency in full-stack web development, relational database design, REST API construction, and the practical application of operations research algorithms. We hope that Uniclass Scheduler serves as a valuable contribution to the academic community and a foundation for future enhancements that will make university administration more efficient and data-driven.

References

1. Carter, M. W., & Laporte, G. (1996). Recent Developments in Practical Course Timetabling. Proceedings of the Second International Conference on the Practice and Theory of Automated Timetabling (pp. 3–19). Springer.
2. Nothegger, C., Mayer, A., Chwatal, A., & Raidl, G. R. (2012). Solving the Post Enrolment-based Course Timetabling Problem by Ant Colony Optimisation. Annals of Operations Research, 194(1), 325–339.
3. Google Developers. (2023). OR-Tools Documentation. Retrieved from https://developers.google.com/optimization
4. FastAPI. (2024). FastAPI Framework Documentation. Retrieved from https://fastapi.tiangolo.com
5. React. (2024). React 19 Documentation. Retrieved from https://react.dev
6. SQLAlchemy. (2024). SQLAlchemy Documentation. Retrieved from https://docs.sqlalchemy.org
7. Vite. (2024). Vite Build Tool Documentation. Retrieved from https://vitejs.dev
8. Müller, T. (2009). ITC-2007 Solver Description: A Hybrid Approach. Annals of Operations Research, 172(1), 429–446.
9. Wren, A. (1996). Scheduling, Timetabling and Rostering — A Special Relationship? Proceedings of the 1st International Conference on the Practice and Theory of Automated Timetabling (pp. 46–75). Springer.
10. Python Software Foundation. (2024). Python 3.x Documentation. Retrieved from https://docs.python.org

Appendix

A. Default Administrator Account

The system is seeded with a default administrator account for testing purposes:

```
Username: admin
Password: Admin123!
```

Note: This account must be changed before any production deployment.

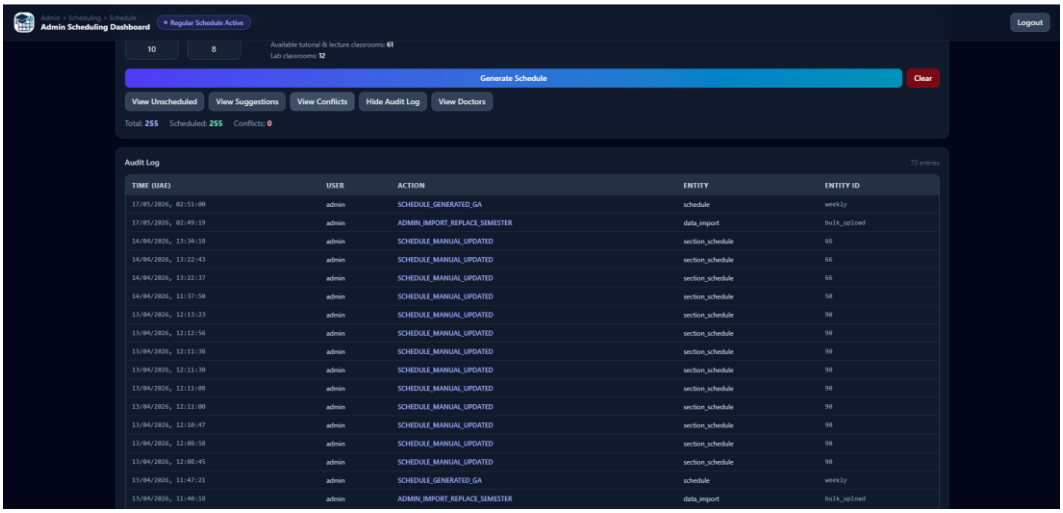
B. API Endpoints Summary

Method	Endpoint	Description
POST	/api/v1/auth/login	Authenticate and retrieve JWT token
GET	/api/v1/courses	List all courses
POST	/api/v1/courses	Create a new course
GET	/api/v1/sections	List all course sections
POST	/api/v1/sections	Create a new section
GET	/api/v1/rooms	List all rooms
POST	/api/v1/rooms	Create a new room
POST	/api/v1/schedules/generate	Trigger automated schedule generation
GET	/api/v1/schedules	Retrieve generated schedule entries
GET	/api/v1/schedules/export/pdf	Export schedule as PDF
GET	/api/v1/schedules/export/docx	Export schedule as DOCX
GET	/api/v1/schedules/export/excel	Export schedule as Excel

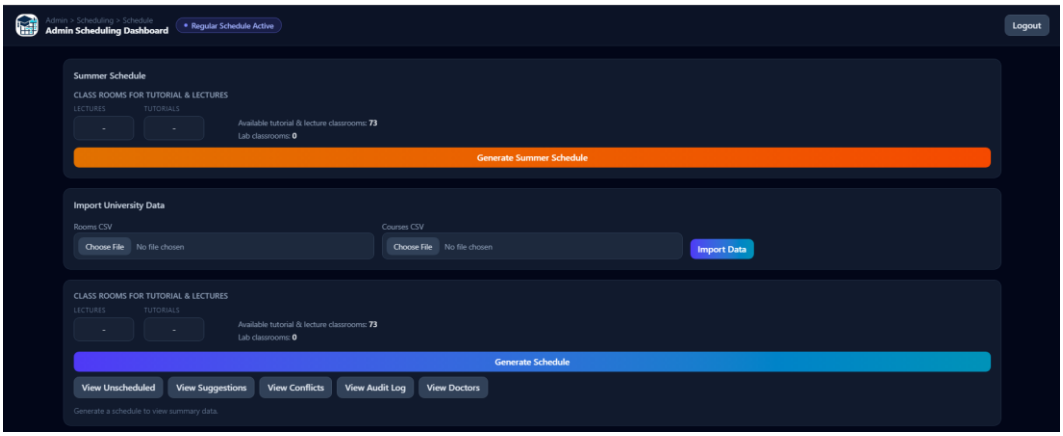
C. Backend Setup Commands

```
# Create virtual environment
py -3 -m venv .venv
.venv\Scripts\Activate.ps1

# Install dependencies
pip install -r requirements.txt

# Run migrations and seed
alembic upgrade head
python scripts/seed_data.py

# Start backend
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

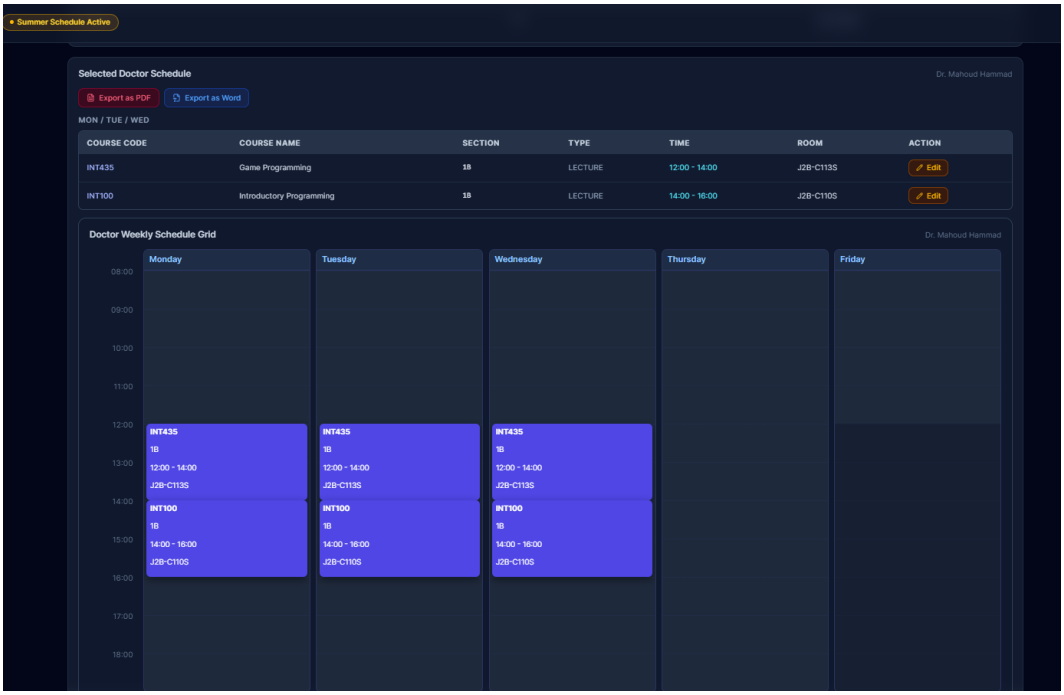
D. Additional Screenshots



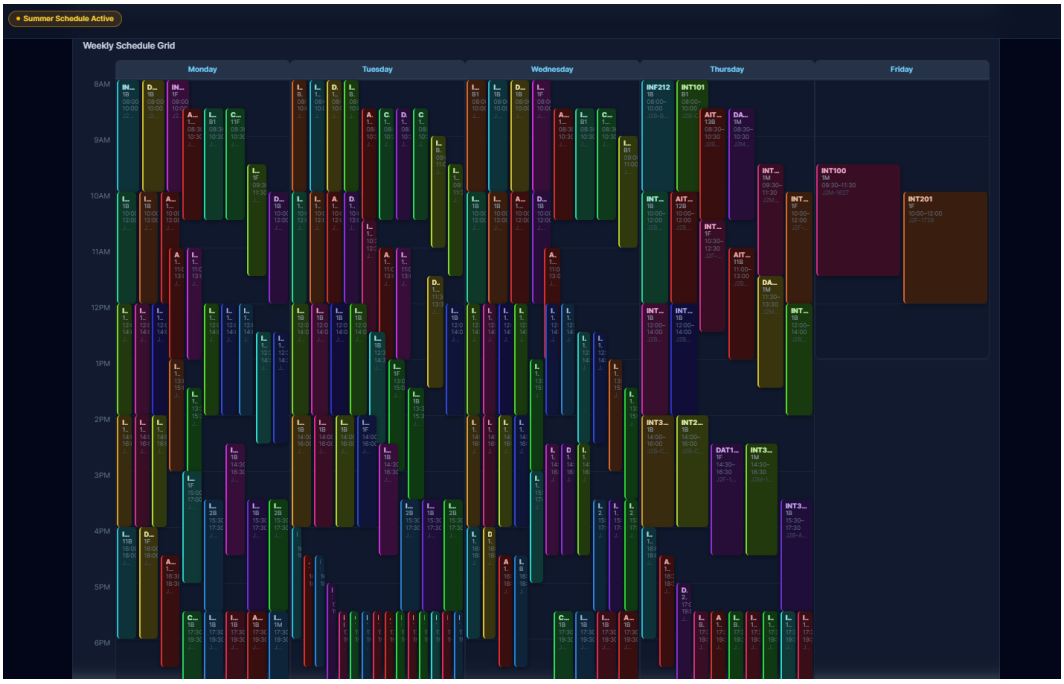
Appendix: Audit Logs Button



Appendix: Admin Dashboard (Alternative View)



Appendix: Instructor Schedule — Summer Session



Appendix: Full Summer Schedule

Doctor Schedule: Dr. Mahoud Hammad

Day	Course Code	Course Name	Section	Type	Time	Room
MON	INT435	Game Programming	1B	LECTURE	12:00 - 14:00	J2B-C113S
MON	INT100	Introductory Programming	1B	LECTURE	14:00 - 16:00	J2B-C110S
TUE	INT435	Game Programming	1B	LECTURE	12:00 - 14:00	J2B-C113S
TUE	INT100	Introductory Programming	1B	LECTURE	14:00 - 16:00	J2B-C110S
WED	INT435	Game Programming	1B	LECTURE	12:00 - 14:00	J2B-C113S
WED	INT100	Introductory Programming	1B	LECTURE	14:00 - 16:00	J2B-C110S

Appendix: Exported Summer PDF

Doctor Schedule - Dr. Mahoud Hammad

Course Code	Course name	Sec	Gen	Mon	Tue	Wed	Thu	Fri	Hall	Credit Hours
INT100	Introductory Programming	1B	B	14:00 - 16:00	14:00 - 16:00	14:00 - 16:00			J2B-C110S	3
INT435	Game Programming	1B	B	12:00 - 14:00	12:00 - 14:00	12:00 - 14:00			J2B-C113S	3

Total Credit Hours	6

Office Hours		Mon	Tue	Wed	Thu	Fri
	Male					
	Female					

Appendix: Exported Summer Word Document

A1 College code

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	College co	College de	Subject co	Course nu	Course tit	Credit hou	Schedule t	Crn	Section nu	Max capa	Day	Meeting ti	Room	Instructor	Merged
2	En	college of	INT	311	Advanced	3	lecture	40000	1B	50	MON	12:30 - 14:	J2B-C030S	Dr. Atta u	yes
3	En	college of	INT	311	Advanced	3	lecture	40000	1B	50	TUE	12:30 - 14:	J2B-C030S	Dr. Atta u	yes
4	En	college of	INT	311	Advanced	3	lecture	40000	1B	50	WED	12:30 - 14:	J2B-C030S	Dr. Atta u	yes
5	En	college of	AIT	111	Artificial I	3	lecture	40021	13B	52	MON	08:30 - 10:	J2B-C110S	TBA	yes
6	En	college of	AIT	111	Artificial I	3	lecture	40023	12B	50	MON	10:00 - 12:	J2B-C030S	TBA	yes
7	En	college of	AIT	111	Artificial I	3	lecture	40024	11B	48	MON	11:00 - 13:	J2B-A220S	TBA	yes
8	En	college of	AIT	111	Artificial I	3	lecture	40026	14B	48	MON	16:30 - 18:	J2B-C113S	TBA	yes
9	En	college of	AIT	111	Artificial I	3	lecture	40027	1B	52	MON	17:30 - 19:	J2B-C110S	TBA	yes
10	En	college of	AIT	111	Artificial I	3	lecture	40021	13B	52	TUE	08:30 - 10:	J2B-C110S	TBA	yes
11	En	college of	AIT	111	Artificial I	3	lecture	40023	12B	50	TUE	10:00 - 12:	J2B-C030S	TBA	yes
12	En	college of	AIT	111	Artificial I	3	lecture	40024	11B	48	TUE	11:00 - 13:	J2B-A220S	TBA	yes
13	En	college of	AIT	111	Artificial I	3	lecture	40026	14B	48	TUE	16:30 - 18:	J2B-C113S	TBA	yes
14	En	college of	AIT	111	Artificial I	3	lecture	40027	1B	52	TUE	17:30 - 19:	J2B-C110S	TBA	yes
15	En	college of	AIT	111	Artificial I	3	lecture	40021	13B	52	WED	08:30 - 10:	J2B-C110S	TBA	yes
16	En	college of	AIT	111	Artificial I	3	lecture	40023	12B	50	WED	10:00 - 12:	J2B-C030S	TBA	yes
17	En	college of	AIT	111	Artificial I	3	lecture	40024	11B	48	WED	11:00 - 13:	J2B-A220S	TBA	yes
18	En	college of	AIT	111	Artificial I	3	lecture	40026	14B	48	WED	16:30 - 18:	J2B-C113S	TBA	yes
19	En	college of	AIT	111	Artificial I	3	lecture	40027	1B	52	WED	17:30 - 19:	J2B-C110S	TBA	yes
20	En	college of	AIT	111	Artificial I	3	lecture	40021	13B	52	THU	08:30 - 10:	J2B-C110S	TBA	yes
21	En	college of	AIT	111	Artificial I	3	lecture	40023	12B	50	THU	10:00 - 12:	J2B-C030S	TBA	yes
22	En	college of	AIT	111	Artificial I	3	lecture	40024	11B	48	THU	11:00 - 13:	J2B-A220S	TBA	yes
23	En	college of	AIT	111	Artificial I	3	lecture	40026	14B	48	THU	16:30 - 18:	J2B-C113S	TBA	yes

Appendix: Summer Timetable Grid